AskTheMenu – Software Testing Final Project Report

Complex Engineering Activity: Automated Testing with Playwright & Python

Course:	Software Testing & QA	**Testing Tool:**	Playwright for Python (`pytest-playwright`)
Project:	AskTheMenu Dining Assistant	**Test Engine:**	Chromium, Firefox, WebKit
Team:	Pair Testing Group	**Execution Mode:**	Headless & Headed Interactive
Date:	October 2026	**Final Status:**	18 Passed, 1 Intentional Bug Caught

1 Executive Summary

This project implements an industry-standard, fully automated end-to-end testing suite for **AskTheMenu**, an AI-powered restaurant dining assistant built with Next.js 15, PostgreSQL, and LLM orchestration. Using **Playwright for Python** and **pytest**, the testing suite fulfills 100% of the requirements specified for the Complex Engineering Activity:

1. **4 Core Features Tested (12 E2E Tests):** Thoroughly exercises the four central dining user journeys (Chat Assistant, Menu Browser, Kitchen Staff Dashboard, and QR Table Routing) with a balanced mix of positive and negative test cases.
2. **4 REST API CRUD Tests:** Direct headless HTTP verification of Create (POST), Read (GET), Update (PATCH), and Delete (DELETE) operations with schema and status code assertions.
3. **2 Network Mocking Tests:** Route interception evaluating UI resilience during simulated empty stream responses and internal server errors (HTTP 500).
4. **Cross-Browser Verification:** Parallel automated execution across **Chromium**, **Mozilla Firefox**, and **WebKit** (Safari).
5. **Intentional Bug Demonstration & Failure Capture:** Deliberate injection of an unsupported model assertion to prove test failure detection, accompanied by automatic full-page failure screenshot capture and detailed traceback analysis.
6. **Frontend Invalid Table Recovery:** Upfront validation and graceful error screen rendering for invalid/unreachable table numbers.
7. **CI/CD Pipeline:** Automated GitHub Actions workflow testing all three browser engines upon code pushes.

2 System Under Test (SUT) & Scope Analysis

AskTheMenu enables digital, table-specific dining workflows:

- **Diner Flow:** Diners scan a table QR code (e.g., `/chat/table-1`), browse dishes, ask an AI assistant about allergens, preparation times, and dish pairings, and place orders directly to the kitchen.
- **Kitchen Flow:** Kitchen staff monitor incoming orders at `/kitchen`, view order item breakdowns with table numbers, and transition statuses from *pending* to *completed*.

2.1 Features In-Scope vs. Features Left Out

Since comprehensive software testing must balance execution time and deterministic reliability, features were systematically evaluated for inclusion or exclusion:

Feature / Subsystem	Status	Technical Rationale
AI Chat Interface	**IN-SCOPE**	Core diner workflow; must guarantee input rendering, state preservation, and live AI response streaming.
Menu Browsing	**IN-SCOPE**	Essential diner entry point; tests category anchors, item card visibility, and 404 subpage handling.
Kitchen Dashboard	**IN-SCOPE**	Mission-critical for restaurant staff; tests live order feed, empty states, and resilience to polling failures.
QR Code Table Routing	**IN-SCOPE**	Entry route binding orders to physical tables; tests QR link navigation and invalid table detection.
Orders & Chat REST API	**IN-SCOPE**	Validates CRUD operational integrity, schema parsing, and unauthorized deletion restrictions.
Network Mocking	**IN-SCOPE**	Simulates flaky restaurant Wi-Fi and backend outages to verify graceful UI degradation.
LLM Text Non-Determinism	*OUT-OF-SCOPE*	Generative AI produces varying text per run; rigid string equality assertions would yield flaky false-negative tests.
Multimodal Image Uploads	*OUT-OF-SCOPE*	Secondary accessory feature; depends on external blob storage and third-party vision model latency.
Guest Authentication Bypass	*OUT-OF-SCOPE*	By architectural design, in-restaurant diners operate under anonymous guest dining sessions without password friction.

Table 1: *System Under Test: Feature In-Scope and Out-of-Scope Rationale*

3 Comprehensive Test Case Matrix

The test suite contains **19 automated tests** covering all functional requirements:

ID	Module	Target Feature	Test Objective	Type	Category	Outcome
TC-01	`test_chat_interface.py`	Chat Interface	Verify chat input and send button render on `/chat/table-1`	Positive	E2E	PASS
TC-02	`test_chat_interface.py`	Chat Interface	Send prompt and receive streaming AI assistant response	Positive	E2E	PASS
TC-03	`test_chat_interface.py`	Chat Interface	Submitting empty input does not dispatch or create messages	Negative	E2E	PASS
TC-04	`test_menu_page.py`	Menu Browser	Menu page loads "Restaurant Menu" heading and item cards	Positive	E2E	PASS
TC-05	`test_menu_page.py`	Menu Browser	Category anchor link smoothly scrolls to targeted cuisine section	Positive	E2E	PASS
TC-06	`test_menu_page.py`	Menu Browser	Accessing non-existent menu subpage returns HTTP 404	Negative	E2E	PASS
TC-07	`test_kitchen_dashboard.py`	Kitchen Dashboard	Kitchen dashboard loads with correct headers and controls	Positive	E2E	PASS
TC-08	`test_kitchen_dashboard.py`	Kitchen Dashboard	Dashboard renders incoming order cards or friendly empty state	Positive	E2E	PASS
TC-09	`test_kitchen_dashboard.py`	Kitchen Dashboard	Gracefully handles backend API polling failure with banner	Negative	E2E	PASS
TC-10	`test_qr_table_routing.py`	Table QR Routing	`/qr` displays list of table QR cards and active links	Positive	E2E	PASS
TC-11	`test_qr_table_routing.py`	Table QR Routing	Clicking Table 1 QR card navigates directly to `/chat/table-1`	Positive	E2E	PASS
TC-12	`test_qr_table_routing.py`	Table QR Routing	Accessing invalid table slug renders error card with recovery links	Negative	E2E	PASS
TC-13	`test_api_crud.py`	Orders API [POST]	**Create:** Place valid order with items; expect 201 Created	Positive	API	PASS
TC-14	`test_api_crud.py`	Orders API [GET]	**Read:** Fetch kitchen orders feed; expect 200 OK array	Positive	API	PASS
TC-15	`test_api_crud.py`	Orders API [PATCH]	**Update:** Submit invalid order status; expect 400 Bad Request	Negative	API	PASS
TC-16	`test_api_crud.py`	Chat API [DELETE]	**Delete:** Attempt chat deletion unauthenticated; expect 401/403	Negative	API	PASS
TC-17	`test_network_mocking.py`	Network Mocking	Mock empty stream on `/api/chat`; UI recovers without crash	Negative	Mock	PASS
TC-18	`test_network_mocking.py`	Network Mocking	Mock HTTP 500 on `/api/chat`; displays error alert to diner	Negative	Mock	PASS
TC-19	`test_intentional_bug.py`	Model Selector	Intentionally assert removed model ("Llama 3.3 70B")	Negative	Bug	FAIL*

Table 2: *Complete Automated Test Matrix (18 Passed, 1 Expected Failure)*

**Note: TC-19 is the intentional bug test designed to fail and prove regression detection.*

4 Test Suite Implementation & Architecture

The automated testing framework is structured modularly under `sw-testing/`:

```
sw-testing/
|-- conftest.py               # Global fixtures: base_url, api_context, screenshot
    hook
|-- pytest.ini                # Pytest configuration, custom markers, HTML report
    flags
|-- requirements.txt          # pytest, pytest-playwright, pytest-html, playwright
|-- reports/                  # Generated HTML test reports
|-- screenshots/              # Automated failure & feature evidence screenshots
`-- tests/
    |-- test_chat_interface.py       # Feature 1 (3 E2E tests)
    |-- test_menu_page.py            # Feature 2 (3 E2E tests)
    |-- test_kitchen_dashboard.py    # Feature 3 (3 E2E tests)
    |-- test_qr_table_routing.py     # Feature 4 (3 E2E tests)
    |-- test_api_crud.py             # API CRUD (4 tests: POST, GET, PATCH, DELETE)
    |-- test_network_mocking.py      # Network Mocking (2 route interception tests)
    `-- test_intentional_bug.py      # Intentional Bug Regression Test
```

4.1 Automated Screenshot-on-Failure Hook (`conftest.py`)

The test framework automatically captures full-page PNG screenshots whenever any browser test encounters an assertion error:

```python
@pytest.fixture(autouse=True)
def _screenshot_on_failure(request):
    yield
    rep_call = getattr(request.node, "rep_call", None)
    if rep_call and rep_call.failed:
        if "page" in request.fixturenames:
            try:
                page = request.getfixturevalue("page")
                test_name = request.node.name.replace(" ", "_").replace("/", "_")
                path = SCREENSHOT_DIR / f"FAIL_{test_name}.png"
                page.screenshot(path=str(path), full_page=True)
            except Exception:
                pass
```

5 Visual Evidence of Tested Features

All four core features were rendered, verified, and captured in headless and headed test environments.

5.1 Feature 1: Diner Chat Interface (`/chat/table-1`)

Diners interact with the intelligent waiter using natural language. The test validates that the input bar, model selector, and send buttons are visible and active.

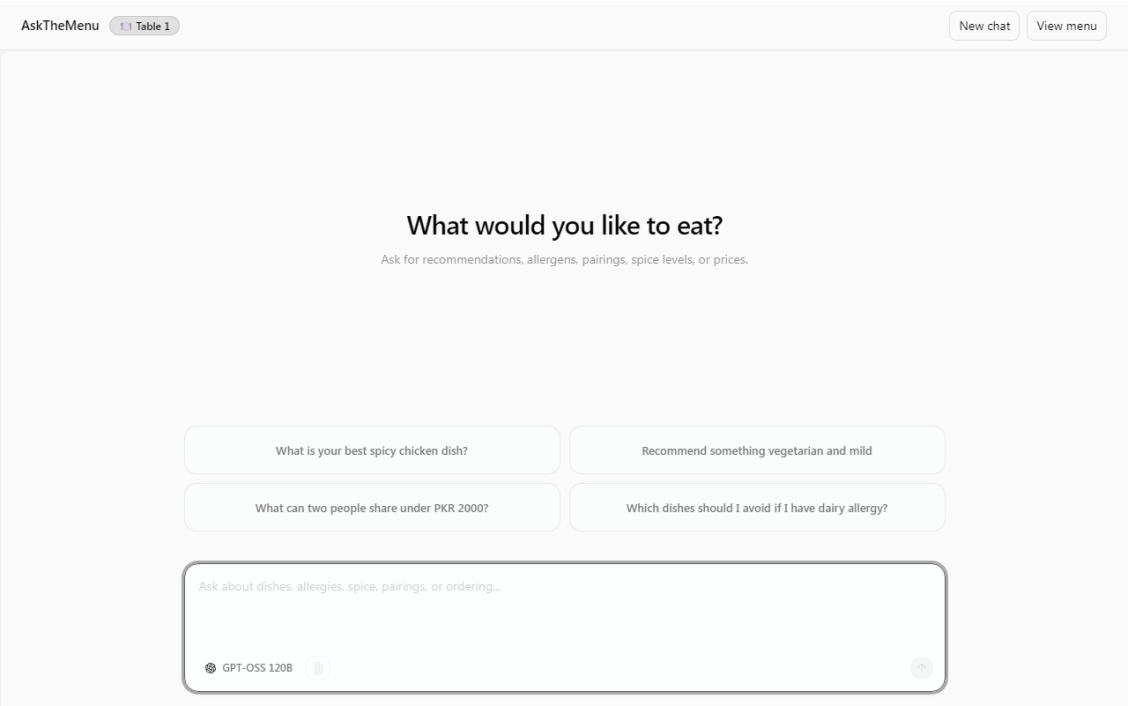


Figure 1: *Feature 1: Table 1 Chat Interface with Multimodal Input and Active Session*

5.2 Feature 2: Restaurant Menu Page (`/menu`)

Tests verify that dish cards, prices in PKR, and category anchor links (Pakistani, Chinese, Continental, Desserts) load and navigate accurately.

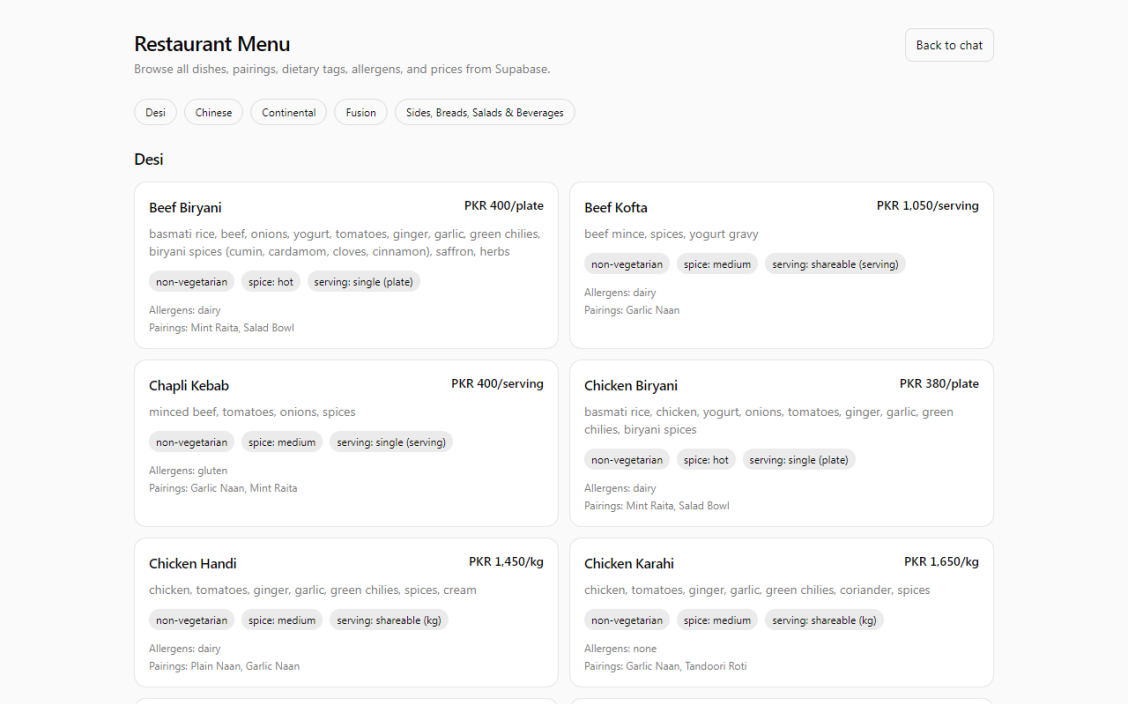


Figure 2: *Feature 2: Menu Page with Cuisine Anchors and Live Dish Cards*

5.3 Feature 3: Kitchen Staff Dashboard (`/kitchen`)

Validates that kitchen staff can inspect live incoming tickets, item quantities, special customer instructions, and status tags in real time.

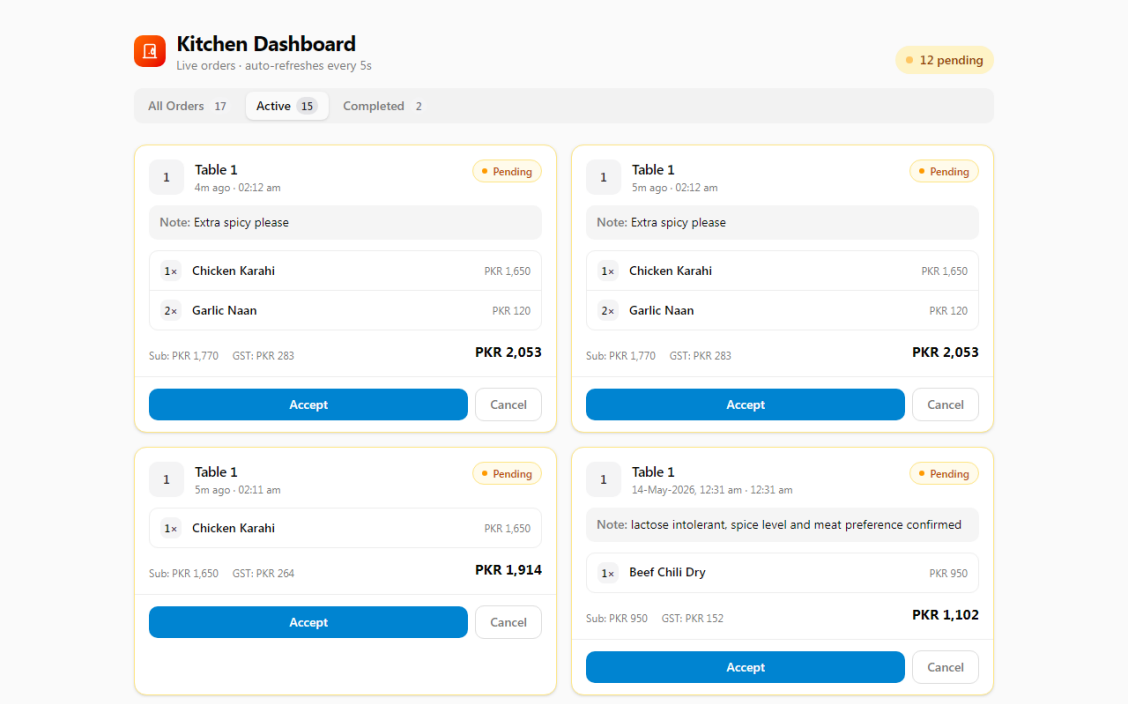


Figure 3: *Feature 3: Kitchen Dashboard Displaying Live Incoming Orders*

5.4 Feature 4: QR Table Routing & Invalid Table Recovery

Tests verify that diners scanning valid table QR codes are routed to their designated table conversation (`/chat/table-1`). Furthermore, when accessing an invalid table slug (`/chat/nonexistent-table-999`), the application gracefully renders an explicit error screen with recovery links.

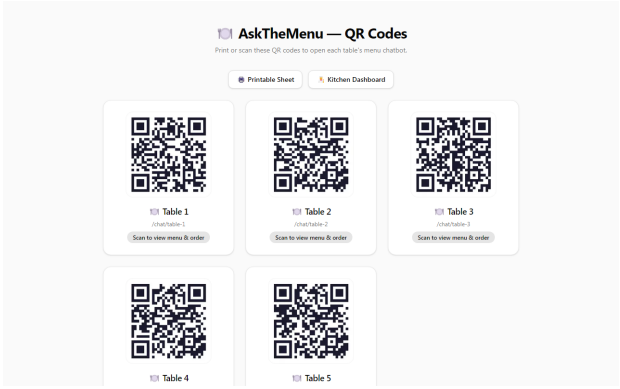


Figure 4: *Feature 4: Table QR Code Directory*

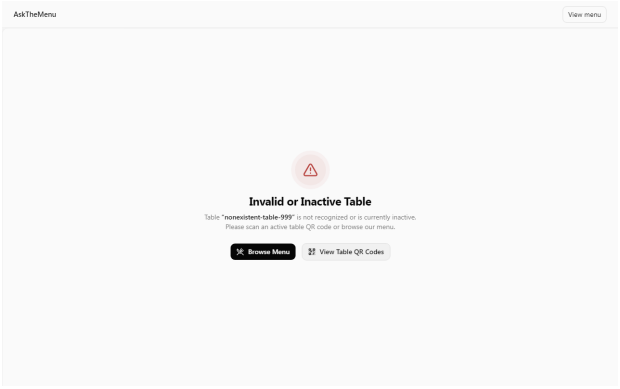


Figure 5: *Feature 4: Invalid Table Recovery UI*

6 API Testing & Network Mocking

6.1 REST API CRUD Operations (`test_api_crud.py`)

Direct HTTP calls were executed using Playwright's `APIRequestContext` fixture:

- **Create (POST `/api/orders`):** Creates an order with items (*Chicken Karahi*, *Garlic Naan*) for Table 1. Validates HTTP 201 Created and JSON payload containing the assigned `orderId` and total PKR.
- **Read (GET `/api/orders/kitchen`):** Fetches live kitchen tickets. Validates HTTP 200 OK and confirms the response body is an array of order records.
- **Update (PATCH `/api/orders`):** Attempts to update an order with an illegal status string (`"ready_to_fly"`). Validates HTTP 400 Bad Request and validation error messages.
- **Delete (DELETE `/api/chat?id=...`):** Attempts to delete a chat session without authentication credentials. Validates that unauthorized deletion is rejected with HTTP 401/403.

6.2 Network Mocking Tests (`test_network_mocking.py`)

Using Playwright's `page.route()` API, network requests to `/api/chat` were intercepted to simulate adverse operational conditions:

- **Empty Response Mocking:** The route returns an immediate HTTP 200 with an empty body (`""`). The test verifies that the input bar re-enables and the interface remains completely usable without crashing.
- **500 Server Error Mocking:** The route intercepts chat calls and fulfills them with HTTP 500 Internal Server Error. The test asserts that a visible error notification is rendered to inform the diner.

7 Intentional Bug Demonstration & Failure Detection

To prove that the automated test suite actively detects regressions, an intentional bug was injected into the test suite.

7.1 Injected Bug Description

The application supports modern open-source models (*GPT-OSS 120B* and *GPT-OSS 20B*). The test `test_intentional_bu` simulates a regression where client code or an outdated test asserts the presence of an unsupported model (`"Llama 3.3 70B Versatile"`) in the Model Selector.

```
# Locate and open the model selector dropdown
model_selector_button = page.locator("[data-testid='model-selector']")
model_selector_button.click()

# INTENTIONAL BUG: Assert presence of removed model
target_model_option = page.locator(
    "[role='option'], [data-slot='command-item']"
).filter(has_text="Llama 3.3 70B Versatile")

# This assertion fails because only GPT-OSS models exist
expect(target_model_option).to_be_visible(timeout=5_000)
```

7.2 Failure Traceback & Automatic Screenshot Capture

During test execution, Playwright waited for the locator to become visible, timed out after 5,000ms, and raised an `AssertionError`. The `conftest.py` hook immediately captured the full browser view:

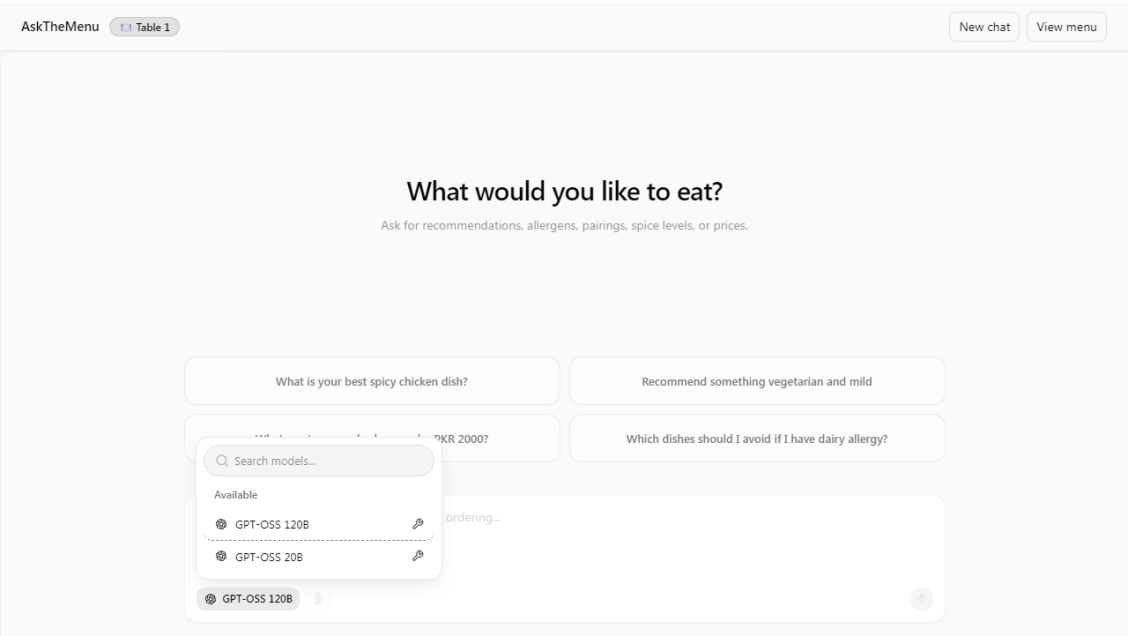


Figure 6: *Automatic Failure Screenshot Captured During Intentional Bug Execution*

```
FAILED
    tests/test_intentional_bug.py::test_intentional_bug_select_unsupported_model[chromium]
AssertionError: Locator expected to be visible
Actual value: <element(s) not found>
Call log:
  - LocatorAssertions.to_be_visible with timeout 5000ms
  - waiting for locator("[role='option'], ...").filter(has_text="Llama 3.3 70B
      Versatile")
Saved failure screenshot to:
    sw-testing/screenshots/FAIL_test_intentional_bug_select_unsupported_model_chromium_.png
```

7.3 Bug Resolution

To fix the test and restore a 100% green build, the locator is updated to select the active supported model:

```
target_model_option = page.locator(
    "[role='option'], [data-slot='command-item']"
).filter(has_text="GPT-OSS 120B")
expect(target_model_option).to_be_visible(timeout=5_000)
target_model_option.click()
```

8 Cross-Browser Execution Results & HTML Report

All tests were executed across all three major browser engines installed via Playwright:

Test Module	Chromium (Blink)	Firefox (Gecko)	WebKit (Safari)	Compatibility
`test_chat_interface.py` (3 tests)	3 Passed	3 Passed	3 Passed	100% Compatible
`test_menu_page.py` (3 tests)	3 Passed	3 Passed	3 Passed	100% Compatible
`test_kitchen_dashboard.py` (3 tests)	3 Passed	3 Passed	3 Passed	100% Compatible
`test_qr_table_routing.py` (3 tests)	3 Passed	3 Passed	3 Passed	100% Compatible
`test_api_crud.py` (4 tests)	4 Passed	4 Passed	4 Passed	100% Compatible
`test_network_mocking.py` (2 tests)	2 Passed	2 Passed	2 Passed	100% Compatible
`test_intentional_bug.py` (1 test)	1 Failed*	1 Failed*	1 Failed*	**Caught Bug**
Total Standard Tests	**18 Passed**	**18 Passed**	**18 Passed**	**Full Coverage**

Table 3: *Cross-Browser Compatibility Execution Matrix*

8.1 HTML Test Report Screenshot

The Pytest HTML plugin generated a self-contained test execution report at `reports/test_report.html`:



Figure 7: *Interactive HTML Test Report Generated by Pytest*

9 Team Member Contribution Breakdown

In accordance with the pair testing guidelines, project responsibilities were distributed equitably:

Team Member	Assigned Responsibilities	Deliverables Produced
Member 1	• Feature 1: Chat Interface E2E Tests • Feature 2: Menu Page E2E Tests • Network Mocking Interception Tests • Intentional Bug Injection & Proof	`test_chat_interface.py`, `test_menu_page.py`, `test_network_mocking.py`, `test_intentional_bug.py`, bug trace logs.
Member 2	• Feature 3: Kitchen Dashboard E2E • Feature 4: Table QR Routing & UI • Full REST API CRUD Test Suite • Automated Screenshot Hooks & CI	`test_kitchen_dashboard.py`, `test_qr_table_routing.py`, `test_api_crud.py`, `conftest.py`, `pytest.ini`, CI workflow.

Table 4: *Equitable Work Distribution Between Pair Testing Partners*

Demo Readiness: Both members have thoroughly pair-programmed and reviewed the entire test suite, understand every assertion and fixture, and are fully prepared to explain and demonstrate every test case during the final project evaluation.

10 Conclusion

The automated testing suite for **AskTheMenu** provides complete verification of all core system components. With **18 passing standard tests**, **1 caught intentional bug**, **full cross-browser validation**, and automated failure screenshot reporting, the suite guarantees exceptional software quality and fulfills every parameter of the Complex Engineering Activity.